

15 — User-Picked Baseline (rule D1) + a Found Client Bug

Proves · A user-pinned baseline (rule D1) — and the client bug the import revealed.

Mechanics this scenario turns on

- Baseline rule: **no battery** → **the worst combination** (`count - 1`); **battery active** → **the third from worst** (`Math.Max(0, count - 3)`). It models what the ship is assumed to do today.
- The **Assumed Configuration** table shows Transit's combinations only, in t/h. The Fuel Consumption figure above it is the annual total across **all** modes.
- Level 2 and Level 3 are computed for **Transit only** (decision D4/Q5 — no workbook counterpart elsewhere). Other modes get an empty result, and the pinned-baseline radio does not reach them.

Panels described below · Expected math once row 4 is selected · What the import actually revealed · How to close the test manually

Anything not described here behaves exactly as in `01-excel-baseline` — same plant, same hours; only what this scenario changes is worked through below.

Trust · verified against the reference workbook. These figures are proof.

Read after · scenario 01.

Test 01's vessel and battery; the file carries **baselineIndex: 4** — as if a user had pinned the WORST row (2 ME + SG, 2.6975) as "how we operate today".

Expected math once row 4 is selected

- Baseline = $2.6975 \times 5\,000 = 13\,487.5$ t/yr
- iEMS Savings = $(2.6975 - 2.6572) \times 5\,000 = 201.25$ t/yr = **\$156 976** (vs 21.2 on the default row)
- "Custom baseline — savings may differ from standard estimate" hint visible
- **Battery Benefit = 173.7 — UNCHANGED.** The invariant this test pins: `Benefit = (optimal_B - optimal_A) × hours` contains no baseline term; the radio moves the "before" picture of the iEMS comparison only.

What the import actually revealed (finding #5 — real client bug)

Restoring the profile did NOT apply index 4 — the panel showed the default 3rd-from-worst row. Mechanism (calculator-page.component.ts): `pendingLoadedBaselineIndex` is consumed by the FIRST `onFormChange` after load, but profile restore fires MORE than one form-change event (the vessel catalog fetch patches fields asynchronously); the second event hits the `selectedBaselineIndex = undefined // reset on new user input` branch and wipes the pin. Related symptom seen on test 12: an early fire can also calculate with DEFAULT form values (fuel price 800) and leave mixed-price panels on screen.

Save is correct (the index IS exported); only restore loses it.

How to close the test manually

Click the last row in Assumed Configuration and verify the four numbers above — especially that the green badge does not move.

Takeaway: baseline selection is a display-and-iEMS-comparison concern; the battery's hardware value is measured optimal-vs-optimal and cannot be steered by the radio. Plus one genuine bug booked for fixing.

KSailCalc manual test scenarios — calculation walkthrough · regenerated 2026-08-10 · numbers verified against commit 559aef2 and unchanged by the backend/client refactors (18 golden snapshots byte-identical)